Java EE — Module 5

JSP and Servlet Integration

Detailed Study Notes

Q1. Differentiate between JSP and Servlet.

> Both **JSP (JavaServer Pages)** and **Servlets** are server-side Java technologies used to build dynamic web applications. They are fundamentally related — every JSP page is internally translated into a Servlet by the web container. However, they differ significantly in their **syntax, purpose, and ease of use** for different tasks.

Core idea:

- A **Servlet** is a **Java class** that generates HTML by writing it programmatically using `out.println()`.
- A **JSP** is an **HTML page** with embedded Java code using `<% %>` tags.

Side-by-side comparison — the same page in both technologies:

Servlet version:

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
protected void doGet(HttpServletRequest req,
HttpServletResponse res)
throws IOException {
res.setContentType("text/html");
PrintWriter out = res.getWriter();
String name = req.getParameter("name");
out.println("<!DOCTYPE html><html><body>");
out.println("<h2>Hello, " + name + "!</h2>");
out.println("</body></html>");
}
}
```

JSP version (same output, far less code):

```
<!DOCTYPE html>
<html><body>
<h2>Hello, <%= request.getParameter("name") %>!</h2>
</body></html>
```

Detailed comparison table:

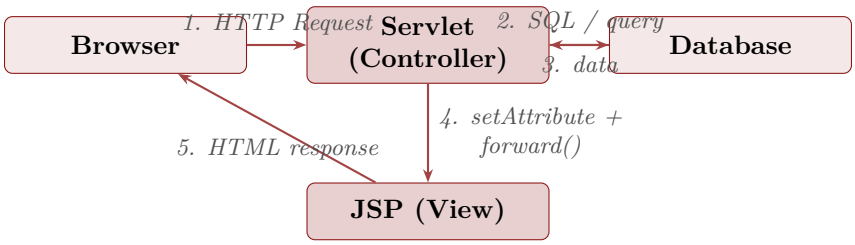
Feature	Servlet	JSP
Nature	Pure Java class	HTML page with embedded Java
Extension	`.java` (compiled to `.class`)	`.jsp` (auto-translated to Servlet)
Primary role	Business logic, control flow	Presentation / view layer
Writing HTML	Tedious — `out.println("<html>")`	Natural — HTML written directly
Writing Java	Natural — plain Java class	Via scriptlets `<% %>`, less clean
Implicit objects	Must declare manually	9 implicit objects available automatically
Compilation	Manually compiled by developer	Auto-compiled by container on first request
Modification	Requires recompile and redeploy	Just save the `.jsp` file; container reloads
Performance	Slightly faster (no translation step)	Small overhead on first request only
Best used for	Controllers, business logic, file I/O	Views, templating, output generation
MVC role	Controller (C)	View (V)

Key insight: JSP and Servlet are not competitors — they are **complementary**. In the MVC pattern, the Servlet is the Controller and JSP is the View. Use each where it excels.

Q2. Explain how JSP and Servlets work together in a web application.

JSP and Servlets work together by dividing responsibilities cleanly: the **Servlet** handles the **logic** (processing requests, querying databases, applying business rules) and the **JSP** handles the **presentation** (rendering HTML to send back to the user). They communicate by sharing data through the `request`, `session`, or `application` scope.

Collaboration flow:



Step-by-step walkthrough — a student login example:

Step 1 — Browser submits a login form to the Servlet:

```
<form action="LoginServlet" method="post">
<input type="text"     name="username" placeholder="Username"/>
<input type="password" name="password" placeholder="Password"/>
<input type="submit" value="Login"/>
</form>
```

Step 2 — Servlet processes the request (Controller):

```
@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {
```

```
@Override
protected void doPost(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {

String username = request.getParameter("username");
String password = request.getParameter("password");

// Business logic --- validate credentials (e.g., check DB)
StudentDAO dao = new StudentDAO();
Student student = dao.findByCredentials(username, password);

if (student != null) {
// Store result in request scope to pass to JSP
request.setAttribute("student", student);
// Forward to success JSP
RequestDispatcher rd =
request.getRequestDispatcher("welcome.jsp");
rd.forward(request, response);
} else {
request.setAttribute("error", "Invalid credentials.");
RequestDispatcher rd =
request.getRequestDispatcher("login.jsp");
rd.forward(request, response);
}
}
}
```

Step 3 — JSP renders the result (View):

```
<%-- welcome.jsp --%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html><body>
<h2>Welcome, ${student.name}!</h2>
<p>Your score: ${student.marks}</p>
</body></html>
```

The shared data mechanism:

Scope	How to set	When to use
`request`	`request.setAttribute()`	Pass data from Servlet to JSP in a single request/forward
`session`	`session.setAttribute()`	Data needed across multiple pages (e.g., logged-in user)
`application`	`context.setAttribute()`	App-wide shared data (e.g., config, counters)

Design principle: Never write SQL or business logic in a JSP, and never write HTML using `out.println()` in a Servlet. The Servlet processes; the JSP displays. This separation makes code maintainable, testable, and readable.

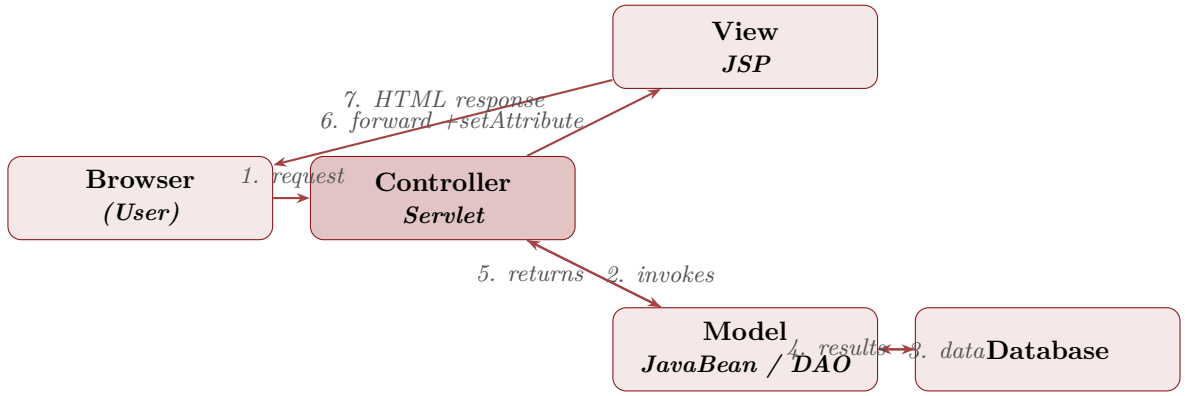
Q3. Explain the Model–View–Controller (MVC) architecture.

MVC (Model–View–Controller) is a software design pattern that separates an application into **three distinct layers**, each with a clearly defined responsibility. This separation makes applications easier to develop, test, maintain, and scale.

The three components:

- **Model** — Represents the **data and business logic**. It encapsulates the application's data (JavaBeans, POJOs), business rules, and database interaction (DAO classes). The Model is completely independent of the UI.
- **View** — Responsible for **presentation**. It displays the data provided by the Model. In Java EE, the View is typically a JSP page that uses EL and JSTL. It contains no business logic.
- **Controller** — Acts as the **intermediary**. It receives user input (HTTP requests), invokes the appropriate Model logic, and selects the correct View to render the response. In Java EE, the Controller is a Servlet.

MVC architecture diagram:



Responsibilities of each layer:

Layer	Responsibility	Java EE technology
Model	Data, business rules, DB access, validation	JavaBeans, POJOs, DAO classes, JDBC
View	Display data, render HTML, no logic	JSP, JSTL, EL, HTML/CSS
Controller	Receive requests, call Model, choose View	Servlet (`HttpServlet`)

Concrete MVC example — Student marks lookup:

Model (JavaBean + DAO):

```
// Student.java --- Model / JavaBean
public class Student {
private int id;
private String name;
private double marks;
// getters and setters ...
}

// StudentDAO.java --- Data Access Object
```

```
public class StudentDAO {
public Student getById(int id) throws SQLException {
// JDBC query to fetch student by ID
Connection con = DBUtil.getConnection();
PreparedStatement ps =
con.prepareStatement("SELECT * FROM students WHERE id=?");
ps.setInt(1, id);
ResultSet rs = ps.executeQuery();
if (rs.next()) {
Student s = new Student();
s.setId(rs.getInt("id"));
s.setName(rs.getString("name"));
s.setMarks(rs.getDouble("marks"));
return s;
}
return null;
}
}
```

Controller (Servlet):

```
@WebServlet("/student")
public class StudentController extends HttpServlet {

@Override
protected void doGet(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {

int id = Integer.parseInt(request.getParameter("id"));

try {
// Call Model
StudentDAO dao = new StudentDAO();
Student student = dao.getById(id);

// Pass data to View
request.setAttribute("student", student);
request.getRequestDispatcher("studentDetail.jsp")
.forward(request, response);

} catch (SQLException e) {
throw new ServletException(e);
}
}
}
```

View (JSP):

```
<%-- studentDetail.jsp --%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html><body>
<c:choose>
<c:when test="${not empty student}">
<h2>Student: ${student.name}</h2>
<p>ID: ${student.id}</p>
<p>Marks: ${student.marks}</p>
```

```
</c:when>
<c:otherwise>
<p>Student not found.</p>
</c:otherwise>
</c:choose>
</body></html>
```

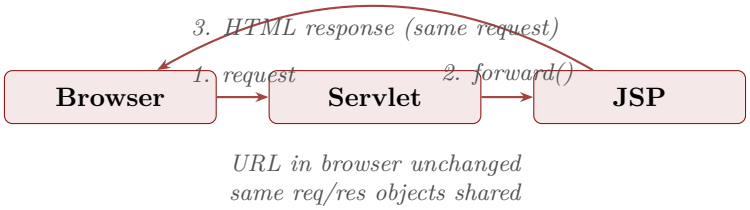
Why MVC? Without MVC, all three concerns (logic, data, presentation) are tangled together in one file — hard to read, test, and change. MVC means a designer can edit the JSP without touching Java, and a developer can change the database logic without touching HTML.

Q4. Explain request forwarding between JSP and Servlet.

Request forwarding is the server-side mechanism by which one resource (a Servlet or JSP) transfers a request to another resource **without the browser knowing**. The browser URL stays the same, and the original `request` and `response` objects are passed intact to the target resource.

It is performed using a `RequestDispatcher` obtained from `HttpServletRequest` or `ServletContext`, then calling `forward()` or `include()`.

Forwarding flow:



Pattern 1 — Servlet forwards to JSP (most common MVC pattern):

```
// ProductServlet.java
@WebServlet("/products")
public class ProductServlet extends HttpServlet {

@Override
protected void doGet(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {

// Fetch product list from DB (Model)
List<Product> products = new ProductDAO().getAllProducts();

// Store in request scope so JSP can read it
request.setAttribute("productList", products);

// Forward to JSP (View) --- same request object carries the data
RequestDispatcher rd =
request.getRequestDispatcher("products.jsp");
rd.forward(request, response);
}
```

```
}
```

```
<%-- products.jsp --%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html><body>
<h2>Product List</h2>
<table border="1">
<tr><th>Name</th><th>Price</th></tr>
<c:forEach var="p" items="${productList}">
<tr>
<td>${p.name}</td>
<td>${p.price}</td>
</tr>
</c:forEach>
</table>
</body></html>
```

Pattern 2 — JSP forwards to Servlet:

A JSP can also forward to a Servlet using a form action or an action element. This is less common but valid.

```
<%-- search.jsp --%>
<form action="SearchServlet" method="get">
<input type="text" name="query" placeholder="Search..."/>
<input type="submit" value="Search"/>
</form>

<%-- Or programmatic forward from JSP --%>
<%
String role = (String) session.getAttribute("role");
if (role == null) {
%>
<jsp:forward page="LoginServlet" />
<%
}
%>
```

Pattern 3 — Servlet includes JSP output:

```
// Using include() to embed a JSP's output into the Servlet's response
@WebServlet("/dashboard")
public class DashboardServlet extends HttpServlet {

@Override
protected void doGet(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {

response.setContentType("text/html");
PrintWriter out = response.getWriter();
out.println("<html><body>");

// Include header JSP
request.getRequestDispatcher("header.jsp")
.include(request, response);
```

```
// Main content
out.println("<h1>Dashboard</h1>");

// Include footer JSP
request.getRequestDispatcher("footer.jsp")
.include(request, response);

out.println("</body></html>");
}
}
```

forward() vs sendRedirect() — key difference:

Feature	forward() / include()	sendRedirect()
Processing side	Server-side only	Client-side (browser makes new request)
Browser URL	Unchanged	Changes to target URL
Request object	Same `req`/`res`	New request created
Request attributes	Preserved in target	Lost (new request)
Speed	Faster (no round trip)	Slower (extra HTTP round trip)
Scope	Within same web app only	Can redirect to any URL
Use case	MVC Servlet → JSP	After POST, login redirect, external URL

Post/Redirect/Get pattern: After a POST request (form submission), always use `sendRedirect()` to a GET page. This prevents the browser from resubmitting the form on refresh. Use `forward()` only for read-only GET operations inside the same app.

Q5. Explain the role of MVC in web application design.

The **MVC pattern** plays a foundational role in modern web application design by enforcing **separation of concerns** — each part of the application has one clearly defined job. This makes large applications manageable, testable, and maintainable by teams working in parallel.

How MVC solves real design problems:

Problem 1 — Mixed concerns (spaghetti JSP):

Without MVC, a single JSP file might contain SQL queries, business logic, and HTML all in one place. This is called a *Model 1* architecture.

```
<%-- BAD: everything in one JSP (Model 1) --%>
<%
Connection con = DriverManager.getConnection("jdbc:mysql://...");
Statement  stmt = con.createStatement();
ResultSet  rs   = stmt.executeQuery("SELECT * FROM students");
%>
<table>
<% while(rs.next()) { %>
<tr><td><%= rs.getString("name") %></td></tr>
<% } %>
</table>
```

This is hard to test, hard to change, and impossible for a UI designer to edit safely.

Solution — MVC (Model 2) architecture:

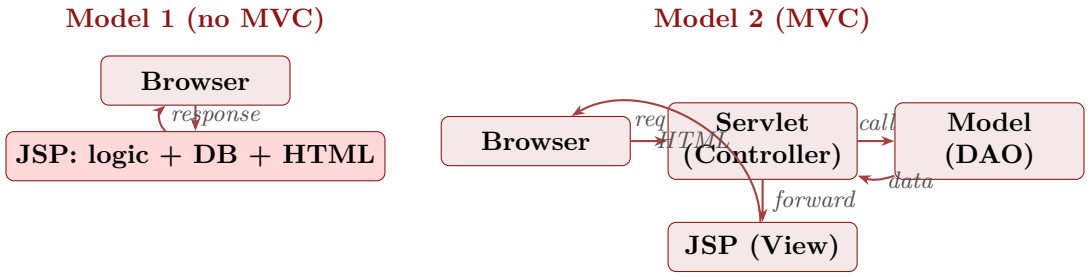
```
// GOOD: Controller (Servlet) handles logic
@WebServlet("/students")
public class StudentController extends HttpServlet {
protected void doGet(HttpServletRequest req,
HttpServletResponse res)
throws ServletException, IOException {
List<Student> list = new StudentDAO().getAll(); // Model
req.setAttribute("students", list);
req.getRequestDispatcher("students.jsp")
.forward(req, res);                          // View
}
}
```

```
<%-- GOOD: View (JSP) only displays --%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<table>
<c:forEach var="s" items="${students}">
<tr><td>${s.name}</td><td>${s.marks}</td></tr>
</c:forEach>
</table>
```

Benefits of MVC in web application design:

- **Separation of concerns** — Model, View, and Controller each have one job. A database change only affects the Model; a layout change only affects the View.
- **Parallel development** — Front-end developers can work on JSP/HTML (View) at the same time as back-end developers work on Servlets and DAOs (Controller and Model), without conflicts.
- **Testability** — The Model (business logic, DAO) can be unit-tested independently of the web layer. No need to start a server to test data logic.
- **Reusability** — The same Model can be used by multiple Views. For example, the same `StudentDAO` can serve a JSP page, a REST API endpoint, and a report generator.
- **Maintainability** — When requirements change, developers know exactly which layer to modify. Adding a new field to the UI only touches the JSP; changing a business rule only touches the Model.
- **Scalability** — Each layer can be scaled independently. Views can be cached; Controllers can be load balanced; Models can be moved to microservices.
- **Security** — Controllers act as gatekeepers. All input validation, authentication checks, and authorization happen in the Servlet before data reaches the View.

Model 1 vs Model 2 (MVC) comparison:



Aspect	Model 1 (JSP only)	Model 2 (MVC)
Architecture	JSP handles everything	Responsibilities separated
Maintainability	Poor — tangled code	Good — clean layers
Testability	Hard — needs browser	Easy — unit test Model independently
Team development	Difficult — conflicts	Easy — parallel development
Scalability	Poor	Good — each layer scalable
Suitable for	Very small, simple projects	Any real-world application

Industry relevance: MVC is not just a Java EE pattern — it is the foundation of virtually every modern web framework: Spring MVC, Django (Python), Ruby on Rails, ASP.NET MVC, Angular, and more. Understanding MVC in Java EE directly transfers to any framework you use in the future.

End of Module 5 — JSP and Servlet Integration